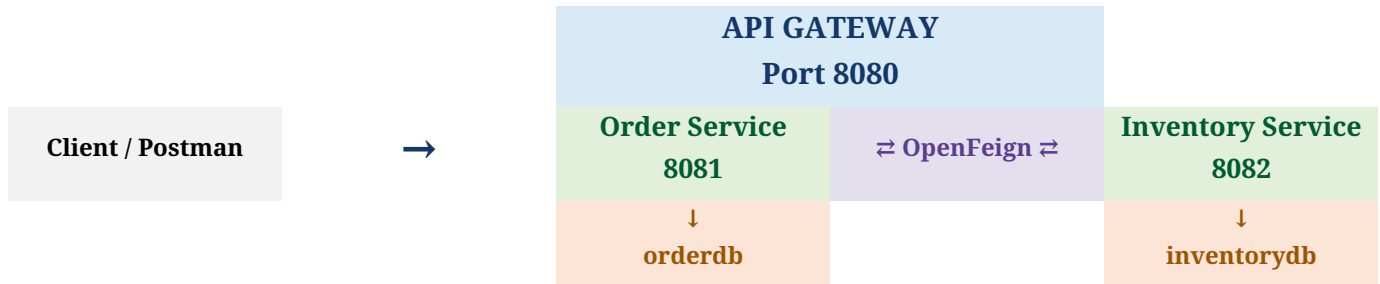


Spring Boot API Gateway

Order Service + Inventory Service + OpenFeign

Detailed Student Guide with Setup, Routing, End-to-End Testing and Troubleshooting



Training setup used in this guide

Item	Value
Order + Inventory workspace	C:\Work\Order-Inventory
API Gateway JAR location	C:\Work\SpringBoot-Microservices\api-gateway\target
API Gateway	http://localhost:8080
Order Service	http://localhost:8081
Inventory Service	http://localhost:8082

1. Learning Objectives

After completing this lesson, students should be able to explain the role of an API Gateway and run the complete Order-Inventory microservice flow from a single public entry point.

- Explain why an API Gateway is used in microservice architecture.
- Understand the difference between client-to-service routing and service-to-service communication.
- Configure gateway routes for Order Service and Inventory Service.
- Run all three applications from their actual local locations.
- Test the gateway using Postman, curl, browser/PowerShell where appropriate.
- Verify that Order Service still calls Inventory Service using OpenFeign.
- Diagnose common 404, 502/503, connection-refused, port and route-configuration problems.

2. Existing Microservice Setup

The application consists of three independently running Spring Boot applications. They do not need to be in the same folder and they do not need to be part of one Maven multi-module project.

Item	Value
api-gateway	Port 8080 • routes external requests • no business database
order-service	Port 8081 • owns orderdb • creates orders
inventory-service	Port 8082 • owns inventorydb • checks product and stock

Important — The physical folder location of a service does not affect HTTP communication. Once a service is running, the gateway communicates with it using host and port, such as `http://localhost:8081`.

3. What Is an API Gateway?

An API Gateway is a single entry point placed between external clients and backend microservices. Instead of requiring a client to know the URL and port of every microservice, the client sends requests to the gateway. The gateway evaluates routing rules and forwards each request to the correct service.

Without Gateway
Client -> Order Service :8081
Client -> Inventory Service :8082
With Gateway
Client -> API Gateway :8080 -> Order Service :8081
-> Inventory Service :8082

- Centralized routing
- Single public base URL
- CORS handling
- Authentication/authorization entry point
- Rate limiting
- Request/response filters

- Observability and correlation IDs
- Resilience and traffic controls

4. API Gateway vs OpenFeign

Concept	API Gateway	OpenFeign
Primary purpose	External request routing	Service-to-service HTTP client
Typical direction	Client → backend service	Order Service → Inventory Service
Used in this project	8080 routes to 8081/8082	Order Service calls Inventory Service on 8082
Should Order Feign call gateway?	No for this basic architecture	Call Inventory Service directly

Key rule — External clients use the API Gateway. Internal Order-to-Inventory communication continues directly through OpenFeign.

5. Gateway Routing Configuration

The gateway must know where to forward each URL pattern. For this local setup, routes use fixed localhost URLs.

```
server:
  port: 8080

spring:
  application:
    name: api-gateway

cloud:
  gateway:
    routes:
      - id: order-service
        uri: http://localhost:8081
        predicates:
          - Path=/api/orders/**

      - id: inventory-service
        uri: http://localhost:8082
        predicates:
          - Path=/api/inventory/**
```

For the configuration above:

- A request beginning with `/api/orders/` is routed to `http://localhost:8081`.
- A request beginning with `/api/inventory/` is routed to `http://localhost:8082`.
- Because we are not using `StripPrefix` or `RewritePath` filters, the path is forwarded unchanged.
- The backend controllers therefore need to expose the same `/api/orders` and `/api/inventory` paths.

Version note — Spring Cloud Gateway 5.x documentation also exposes the `WebFlux` server route property under `spring.cloud.gateway.server.webflux.routes`. If your existing gateway JAR already starts and routes correctly with `spring.cloud.gateway.routes`, keep the configuration that matches your Spring Cloud version.

6. Order Service Internal Feign Configuration

Adding a gateway does not change the existing Order Service → Inventory Service Feign communication for this exercise.

```
inventory.service.url=http://localhost:8082
```

```
@FeignClient(
    name = "inventory-service",
    url = "${inventory.service.url}"
)
public interface InventoryClient {

    @GetMapping("/api/inventory/{productCode}")
    InventoryResponse checkStock(
        @PathVariable("productCode") String productCode,
        @RequestParam("quantity") Integer quantity);
}
```

Do not change this to 8080 for this lesson — The API Gateway is the public client entry point. The Feign client is internal service communication and continues calling Inventory Service directly on port 8082.

7. Exact Local Folder Setup

The services can be started from the following Windows locations.

```
Order + Inventory:\nC:\Work\Order-Inventory\n\nInventory project:\nC:\Work\Order-Inventory\inventory-service\n\nOrder project:
\nC:\Work\Order-Inventory\order-service\n\nAPI Gateway JAR:\nC:\Work\SpringBoot-Microservices\api-gateway\target\api-
gateway-1.0.0.jar
```

Use three terminals so you can see the logs of every application independently.

8. Pre-Test Checklist

- PostgreSQL is running.
- inventorydb exists and contains the inventory table.
- orderdb exists and contains the orders table (or Hibernate is configured to create/update it).
- Inventory Service is configured for port 8082.
- Order Service is configured for port 8081.
- API Gateway is configured for port 8080.
- Order Service property inventory.service.url=http://localhost:8082 is present.
- No other application is already using ports 8080, 8081 or 8082.
- The API Gateway JAR name in the target folder matches the java -jar command.

9. Start the Applications in the Correct Order

9.1 Start Inventory Service First

```
cd C:\Work\Order-Inventory\inventory-service\nmvn spring-boot:run
```

Expected service URL:

```
http://localhost:8082
```

Before starting Order Service, verify Inventory Service independently:

```
GET http://localhost:8082/api/inventory/P1001?quantity=2
```

Why first? — Order Service depends on Inventory Service when a new order is placed. Starting Inventory first avoids an immediate Feign connection failure during testing.

9.2 Start Order Service Second

```
cd C:\Work\Order-Inventory\order-service\nmvn spring-boot:run
```

Expected service URL:

```
http://localhost:8081
```

Confirm the Feign URL:

```
inventory.service.url=http://localhost:8082
```

9.3 Start API Gateway Third

```
cd C:\Work\SpringBoot-Microservices\api-gateway\target\njava -jar api-gateway-1.0.0.jar
```

Expected gateway URL:

```
http://localhost:8080
```

JAR name check — If the generated JAR has a different version or artifact name, use the exact filename shown by `dir` in the target folder.

10. Testing Strategy

Test from the inside out. This makes it easy to identify which layer is failing.

1. Test Inventory Service directly.
2. Test Order Service directly (only for backend verification).
3. Test Inventory route through API Gateway.
4. Test Order route through API Gateway.
5. Verify the Order → Inventory Feign call in logs.

6. Verify the saved order in orderdb.
7. Run failure and negative test cases.

11. Test Inventory Service Directly

This test verifies Inventory Service and inventorydb before involving the gateway.

```
GET http://localhost:8082/api/inventory/P1001?quantity=2
```

Expected Response

```
{
  "productCode": "P1001",
  "productName": "Laptop",
  "availableQuantity": 10,
  "available": true
}
```

If this fails, do not troubleshoot the gateway yet. Fix Inventory Service, database connectivity, seed data, or controller mapping first.

12. Test Order Service Directly (Backend Verification Only)

Before switching completely to gateway URLs, one direct test can verify that Order Service and Feign communication work correctly.

```
POST http://localhost:8081/api/orders
Content-Type: application/json
```

```
{
  "productCode": "P1001",
  "quantity": 2,
  "price": 75000
}
```

During this call, Order Service should make an internal Feign call to:

```
GET http://localhost:8082/api/inventory/P1001?quantity=2
```

After backend verification — For normal client testing, switch to the gateway URLs on port 8080. Direct 8081/8082 calls are useful mainly for diagnostics and isolated service tests.

13. Test Inventory Through API Gateway

```
GET http://localhost:8080/api/inventory/P1001?quantity=2
```

Expected routing:

```
Client -> 8080 -> route /api/inventory/** -> 8082 -> InventoryController -> inventorydb
```

Expected response should be the same business response returned when Inventory Service is called directly.

14. Test Order Through API Gateway

```
POST http://localhost:8080/api/orders
Content-Type: application/json
```

```
{
  "productCode": "P1001",
  "quantity": 2,
  "price": 75000
}
```

Expected end-to-end flow:

8. Postman sends POST /api/orders to API Gateway on 8080.
9. Gateway matches the /api/orders/** route.
10. Gateway forwards the request to Order Service on 8081.
11. OrderController receives the request.
12. OrderService validates the business request.
13. OrderService calls InventoryClient (OpenFeign).
14. Feign sends GET /api/inventory/P1001?quantity=2 directly to Inventory Service on 8082.
15. Inventory Service queries inventorydb and returns availability.
16. Order Service creates and saves the order to orderdb when stock is available.
17. Order Service returns the response to the gateway.
18. Gateway returns the response to Postman.

15. Expected Order Success Response

```
{
  "orderId": 1,
  "productCode": "P1001",
  "quantity": 2,
  "price": 75000,
  "status": "CONFIRMED",
  "createdAt": "2026-08-24T14:30:00"
}
```

The exact orderId and timestamp will vary. The important checks are productCode, quantity, price, status and successful persistence.

16. Which URLs Should Students Use?

Purpose	URL	Use
Normal client order request	http://localhost:8080/api/orders	YES
Normal client inventory request	http://localhost:8080/api/inventory/P1001?quantity=2	YES
Direct Order Service	http://localhost:8081/api/orders	Diagnostics / isolated testing

Direct Inventory Service	http://localhost:8082/api/inventory/P1001?quantity=2	Diagnostics / isolated testing
--------------------------	--	--------------------------------

Student rule — Once the gateway is introduced, Postman should normally use port 8080. Ports 8081 and 8082 remain useful for troubleshooting and service-level testing.

17. Negative and Failure Test Cases

17.1 Product Not Found

```
GET http://localhost:8080/api/inventory/P9999?quantity=2
```

Expected: Inventory Service returns a structured 404 response through the gateway. The gateway should not convert a normal backend 404 business response into success.

17.2 Insufficient Stock

```
POST http://localhost:8080/api/orders
```

```
{
  "productCode": "P1001",
  "quantity": 9999,
  "price": 75000
}
```

Expected: Inventory returns available=false, Order Service does not save the order, and the client receives a meaningful insufficient-stock response.

17.3 Stop Inventory Service

Stop the terminal running Inventory Service, keep Order Service and Gateway running, then call:

```
POST http://localhost:8080/api/orders
```

Expected flow: Gateway reaches Order Service, but the Feign call to 8082 fails. Order Service should map this to an InventoryServiceUnavailableException or equivalent and return a 503-style error response.

17.4 Stop Order Service

Keep Gateway running but stop Order Service. Then call:

```
POST http://localhost:8080/api/orders
```

Expected: the gateway cannot connect to http://localhost:8081. Depending on gateway/version/error handling, the client will receive a gateway/server error such as 5xx. Check the gateway log for the connection-refused root cause.

17.5 Stop API Gateway

Call port 8080 after stopping the gateway. Expected: connection refused because the public entry point is not running.

18. How to Verify the Flow Using Logs

Keep all three terminals visible. For one order request, students should observe activity in this order:

19. API Gateway receives/routes the request.
20. Order Service receives POST /api/orders.
21. Inventory Service receives the Feign GET request.
22. Inventory JPA/Hibernate executes a query against inventorydb.
23. Order Service receives inventory response.
24. Order JPA/Hibernate inserts into orderdb.
25. Client receives the response.

Teaching tip — Ask students to identify which terminal should show each part of the request. This reinforces that the applications are separate processes even when all run on localhost.

19. Verify Database Results

Inventory Data

```
SELECT * FROM inventory;
```

Order Data

```
SELECT * FROM orders ORDER BY id DESC;
```

After a successful order request, a new row should exist in the orders table. In the basic use case, inventory stock may not be deducted unless you implemented stock reservation/deduction separately.

Important design note — Checking stock and saving an order is not the same as reserving stock. Production systems need concurrency and consistency controls if stock must be guaranteed.

20. Optional Command-Line Testing

20.1 curl - Inventory Through Gateway

```
curl "http://localhost:8080/api/inventory/P1001?quantity=2"
```

20.2 curl - Create Order Through Gateway

```
curl -X POST "http://localhost:8080/api/orders" ^  
-H "Content-Type: application/json" ^  
-d '{"productCode":"P1001","quantity":2,"price":75000}'
```

20.3 PowerShell

```
Invoke-RestMethod `   
-Uri "http://localhost:8080/api/orders" `
```

```
-Method Post `
-ContentType "application/json" `
-Body '{"productCode":"P1001","quantity":2,"price":75000}'
```

21. Troubleshooting Guide

Problem	Likely Cause	Check / Fix
8080 connection refused	Gateway not running	Start java -jar; verify port 8080 in logs.
Gateway returns 404	Path predicate does not match	Check /api/orders/** and /api/inventory/** plus controller paths.
Gateway 5xx to orders	Order Service not reachable	Verify http://localhost:8081 and terminal logs.
Order returns inventory unavailable	Inventory Service stopped/wrong Feign URL	Verify inventory.service.url=http://localhost:8082.
Inventory product not found	P1001 row missing	Query inventory table and insert seed data.
Port already in use	Another process owns 8080/8081/8082	Stop conflicting process or change port consistently.
JAR not found	Filename/version different	Run dir in target folder and use exact JAR filename.
Order not saved	Stock unavailable or DB error	Check Order Service logs and orderdb connection.
Gateway starts but routes absent	Wrong config property for framework version	Check Spring Cloud version and effective gateway configuration.

22. Useful Windows Commands

Check which process is using a port:

```
netstat -ano | findstr :8080
netstat -ano | findstr :8081
netstat -ano | findstr :8082
```

Identify the process by PID:

```
tasklist | findstr <PID>
```

Stop a process only when you are sure it is safe to terminate:

```
taskkill /PID <PID> /F
```

23. Understanding the Gateway Route

```
- id: order-service
uri: http://localhost:8081
predicates:
  - Path=/api/orders/**
```

- id is a unique name for the route.
- uri is the backend destination.
- Path is a predicate: the route is selected only when the incoming path matches.
- The gateway proxies the HTTP method, headers, body and path to the destination according to the route/filter configuration.

24. What Would Change in a Production Architecture?

Hard-coded localhost URLs are appropriate for a classroom local setup, but production deployments commonly add:

- Service discovery / platform DNS instead of localhost.
- Load balancing across multiple service instances.
- Authentication and authorization at the gateway.
- TLS/HTTPS.
- Centralized logging, tracing and metrics.
- Rate limiting and request quotas.
- Circuit breakers, timeouts and resilience policies.
- Environment-specific configuration using profiles/config management.

25. Complete Student Testing Checklist

- ☐ PostgreSQL is running.
- ☐ inventory-service starts successfully on 8082.
- ☐ Direct Inventory API returns P1001.
- ☐ order-service starts successfully on 8081.
- ☐ Order Service has inventory.service.url=http://localhost:8082.
- ☐ Direct Order API confirms Feign communication.
- ☐ api-gateway JAR starts successfully on 8080.
- ☐ Gateway Inventory GET succeeds.
- ☐ Gateway Order POST succeeds.
- ☐ Order appears in orderdb.
- ☐ Unknown product test returns expected error.
- ☐ Insufficient-stock test does not create an order.
- ☐ Inventory-down test demonstrates Feign failure handling.
- ☐ Order-down test demonstrates gateway/backend connectivity failure.
- ☐ Students can explain why normal Postman calls use 8080.

26. Interview / Viva Questions

- What problem does an API Gateway solve?
- Why should clients use port 8080 instead of directly using 8081 and 8082?
- What is a gateway route predicate?
- What is the difference between API Gateway and OpenFeign?
- Why does Order Service still call Inventory Service on 8082?
- What happens when Inventory Service is unavailable?
- What happens when Order Service is unavailable but the gateway is running?
- Why are orderdb and inventorydb owned by separate services?
- How would Eureka or Kubernetes change the hard-coded URLs?
- Where would authentication, rate limiting and centralized tracing commonly be added?

27. Final Summary

The API Gateway provides a single entry point on port 8080 for external clients. It routes `/api/orders/**` requests to Order Service on 8081 and `/api/inventory/**` requests to Inventory Service on 8082. Order Service continues using OpenFeign to communicate directly with Inventory Service for stock validation. Each service remains independently deployable and keeps ownership of its own database.

```
Client / Postman
  |
  v
API Gateway :8080
  |      |
  v      v
Order :8081  Inventory :8082
  |          |
  |          |
orderdb    inventorydb
  |
+---- OpenFeign ----> Inventory :8082
```

Remember — Gateway = external routing. Feign = internal service-to-service call. Database per service = ownership boundary.

28. Framework Reference Notes

For instructor reference, Spring documents Spring Cloud Gateway as a routing layer for APIs and provides both WebFlux and Web MVC variants. The traditional Spring Cloud Gateway starter is built on WebFlux/Reactor, while newer documentation also lists server-specific configuration namespaces. Always align dependency and property names with the Spring Boot/Spring Cloud versions used by the project.

<https://docs.spring.io/spring-cloud-gateway/reference/>

<https://docs.spring.io/spring-cloud-gateway/reference/appendix.html>

<https://docs.spring.io/spring-cloud-openfeign/reference/>